

Motion Control Video Editing 调研与复现汇报

— 论文调研 + FYM 复现 —

参考视频 (Reference Video)

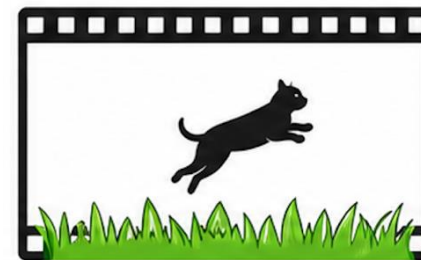


文本提示 (Prompt)

“一只猫在起跳
并落在草地上”
... ..



生成视频 (Generated Video)



可控运动



外观保持



文本对齐



时空一致



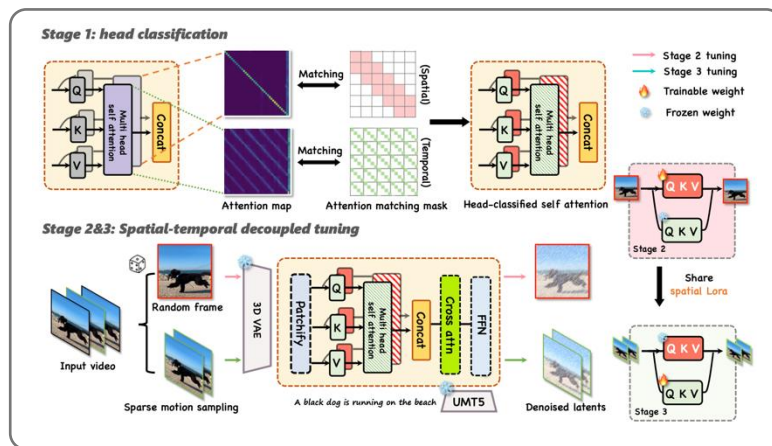
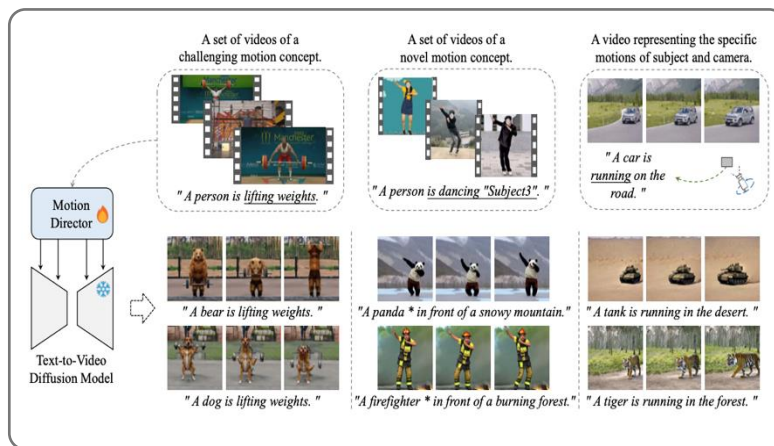
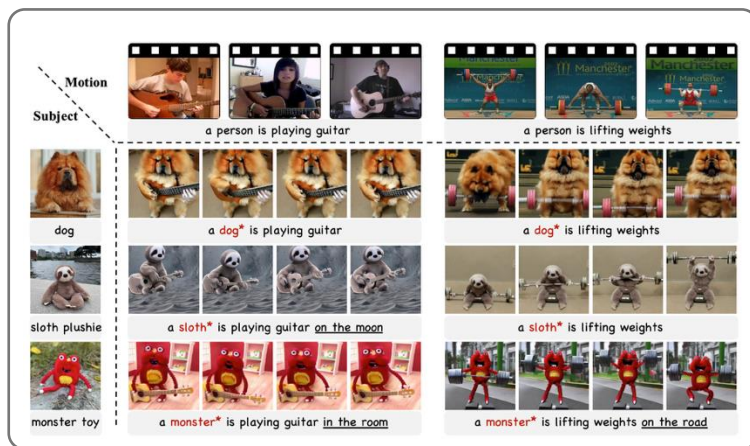
高效推理

调研对象 (10篇论文)

DreamVideo | MotionDirector | CustomTTT | Motion Inversion | DeT |
SMRABooth | FOLLOW-YOUR-MOTION | FASTVMT | DreamRelation | SynMotion

任务定义：我们需要控制什么？

	Customized Video Generation	Video Motion Transfer
输入 (Input)	- 文本提示 - 可选参考图片或视频	- 源视频（提供运动） - 目标主体/场景的载体视频
输出 (Output)	新的视频 (外观 + 运动 由模型生成)	将源运动迁移到目标的视频 (保留目标外观)



关键 takeaway: 从 “生成” 到 “可控”，再聚焦到 “运动可控” 本身。

共同挑战：为什么 motion control 难？



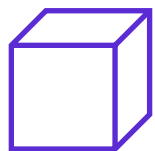
1. Appearance–Motion Entanglement

外观与运动高度耦合，
难以单独控制运动。



2. Motion Representation

缺乏通用、紧凑且可泛化的
运动表示。



3. DiT Spatial– Temporal Decoupling

DiT 在时空建模上存在解耦
与对齐难题。

Video =



Subject



Background



Camera



Motion



Text

4. Evaluation & Efficiency



评测标准不统一，
推理成本高、效率低。



核心难点：如何在保证外观与文本一致的同时，精准、可泛化且高效地控制运动？

路线一： Adapter / LoRA 结构解耦

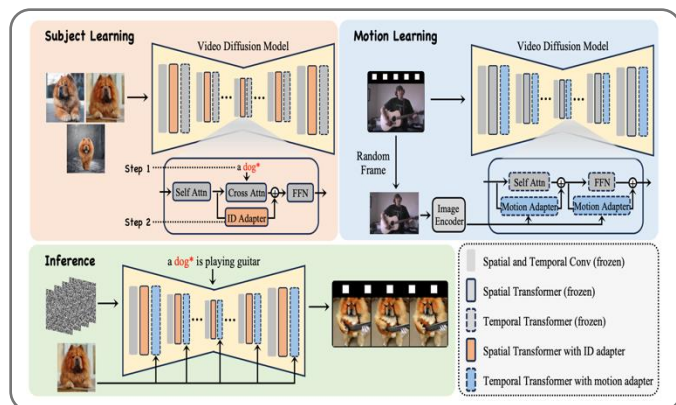
DreamVideo

核心思想：

在预训练扩散模型中插入小型 Adapter，解耦并注入运动信号。

主要策略：

提出运动 Adapter，实现高保真、可控生成。



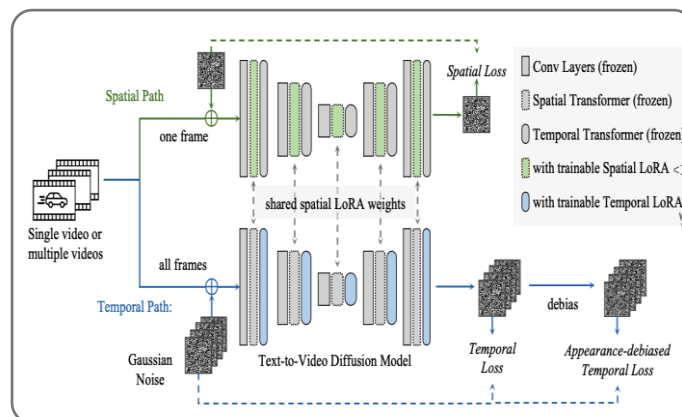
MotionDirector

核心思想：

采用运动模块+身份模块双分支，分离建模运动与外观。

主要策略：

解耦表征 + 模块化设计，提升可控性与泛化性。



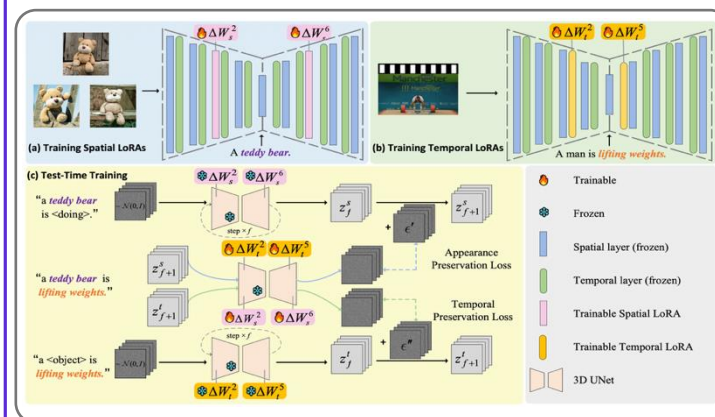
CustomTTT

核心思想：

在时空注意力中注入轻量参数，实现 TTT 可控适配。

主要策略：

CustomTTT 层设计，增强运动保真与效率。

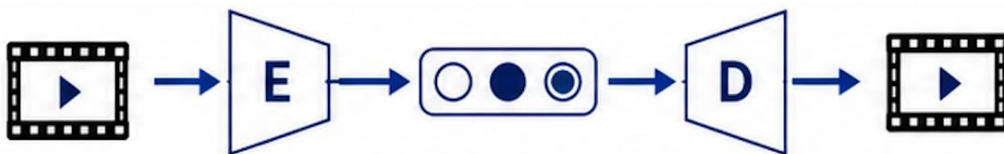


Takeaway: 通过轻量参数模块，在不大幅改动主干的前提下实现运动可控与泛化。

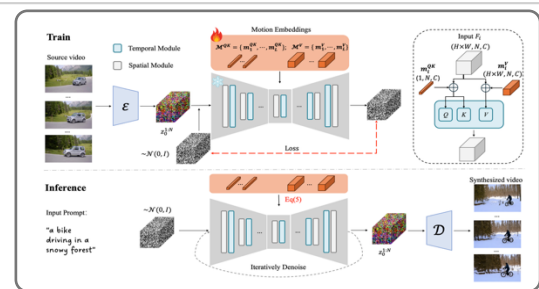
路线二：显式 Motion Representation

Motion Inversion

简化原理图



关键词：
QK/V
embedding

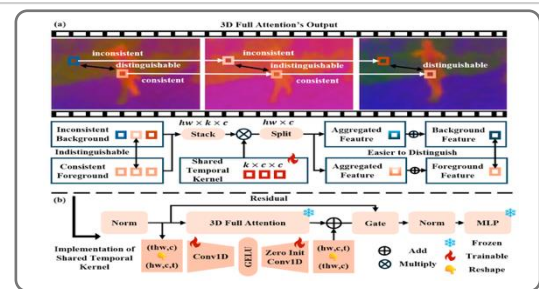


DeT

简化原理图



关键词：
dense tracking
+ tokens

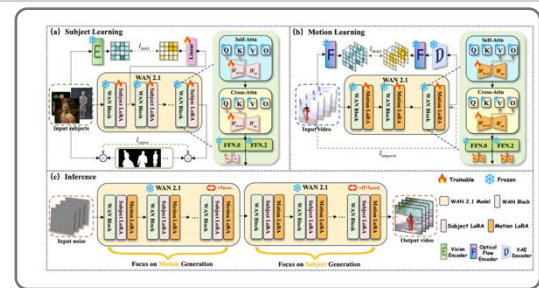


SMRABooth

简化原理图



关键词：
SuRA / MoRA



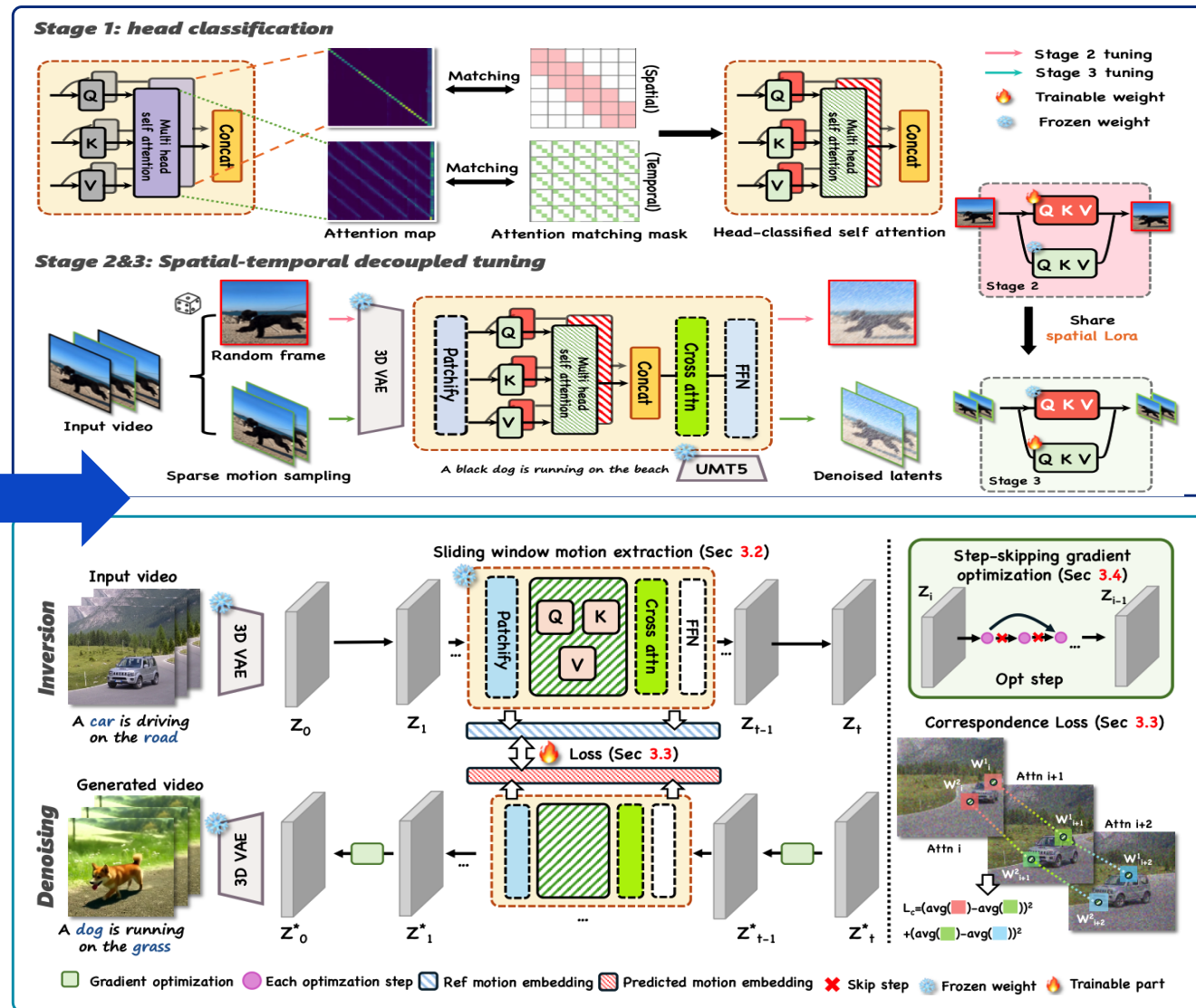
Takeaway: 显式、可泛化的运动表示是高质量迁移的基础。

路线三： DiT 时代的解耦与效率

U-Net vs DiT 对比

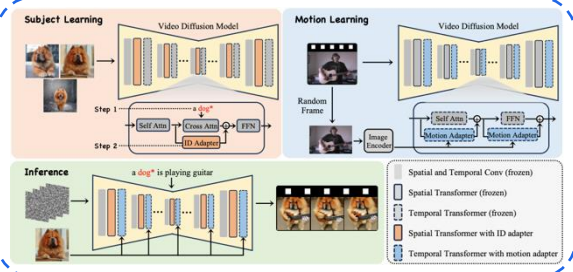
维度	U-Net (传统)	DiT (扩散 Transformer)
结构偏置	卷积 + 下采样, 上下文局部	全局自注意力, 长程建模强
时空建模	耦合较强, 易混淆外观与运动	时空更可分, 但需对齐策略
可扩展性	受限于卷积感受野与分辨率	易扩展, 适合高分辨率视频
计算复杂度	中等	较高 (自注意力/并行优化)
可控性潜力	中等	高 (更易解耦与模块化)

两种代表性解法: Tuning free_fym/FASTVMT

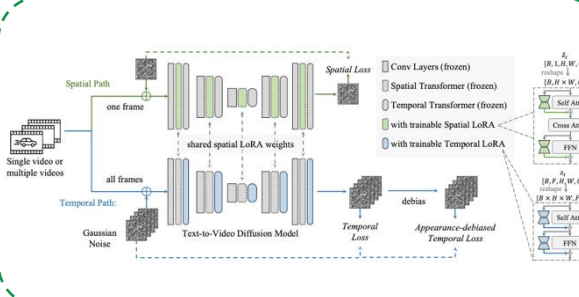
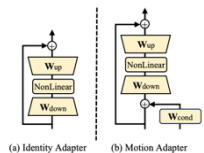


DiT挑战

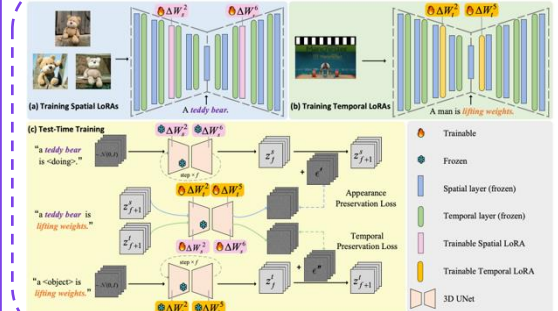
时空对齐难/计算成本高/运动信息注入位置?/如何在推理时实现高效解耦?



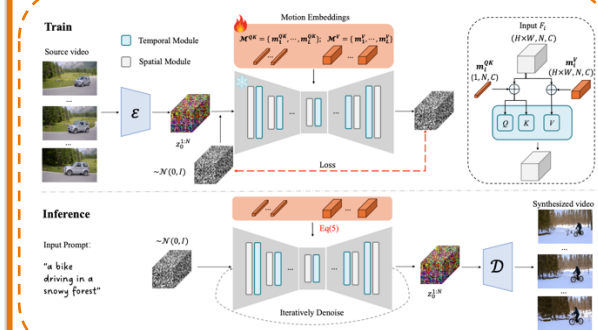
- Subject + Motion 联合定制
- Identity Adapter 与 Motion Adapter 解耦主体与运动



- Dual-path LoRA: Spatial 学外观, Temporal 学运动
- 用 appearance-debiased temporal loss 提升运动泛化



- 关注多 LoRA 合并冲突
- 通过 test-time training 修正适配结果



- 提出显式 Motion Embeddings
- 用 QK/V 级表示增强运动可迁移性

2024 CVPR



DreamVideo

2024 ECCV



MotionDirector

2025 AAAI



CustomTTT

2025 SIGGRAPH



Motion Inversion

2025 ICCV



DeT

2026 CVPR



SMRABooth

2026 ICLR

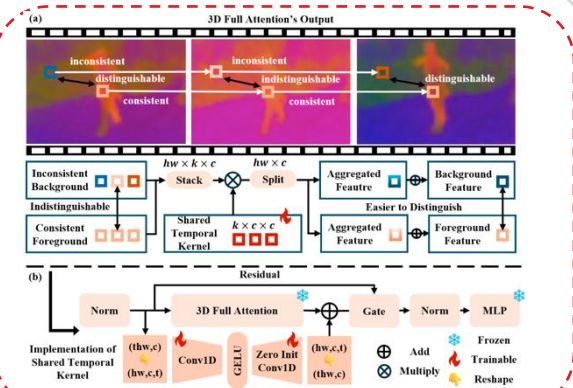


FOLLOW-YOUR-Motion

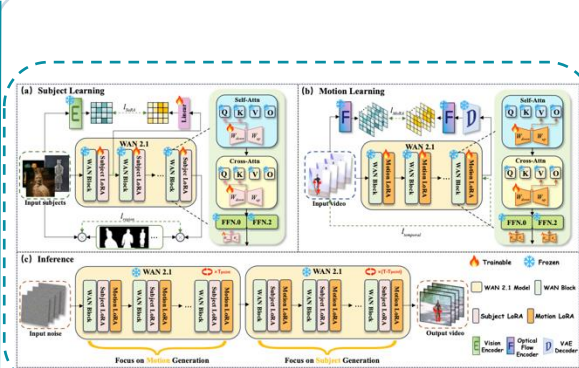
2026 ICLR



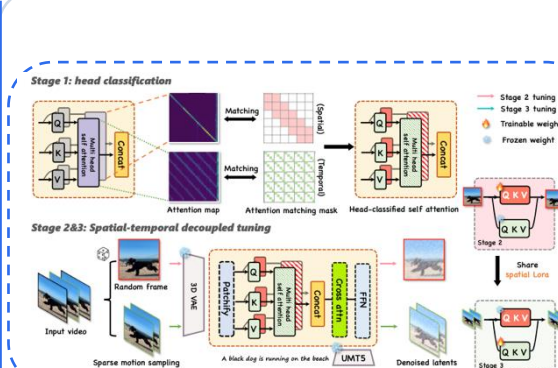
FASTVMT



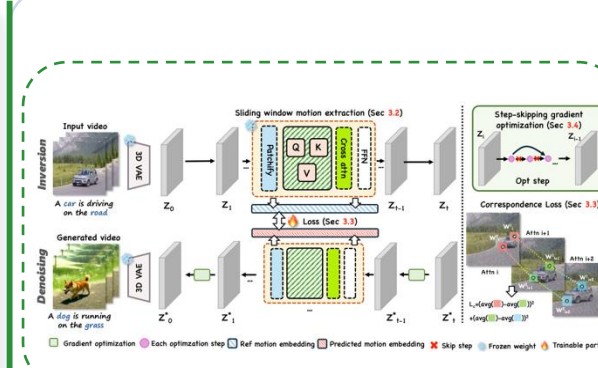
- 面向 DiT 的运动转移与 benchmark
- Dense tracking + temporal kernel 改善运动一致性



- Object-level representation alignment
- SuRA / MoRA 协同对齐主体与运动



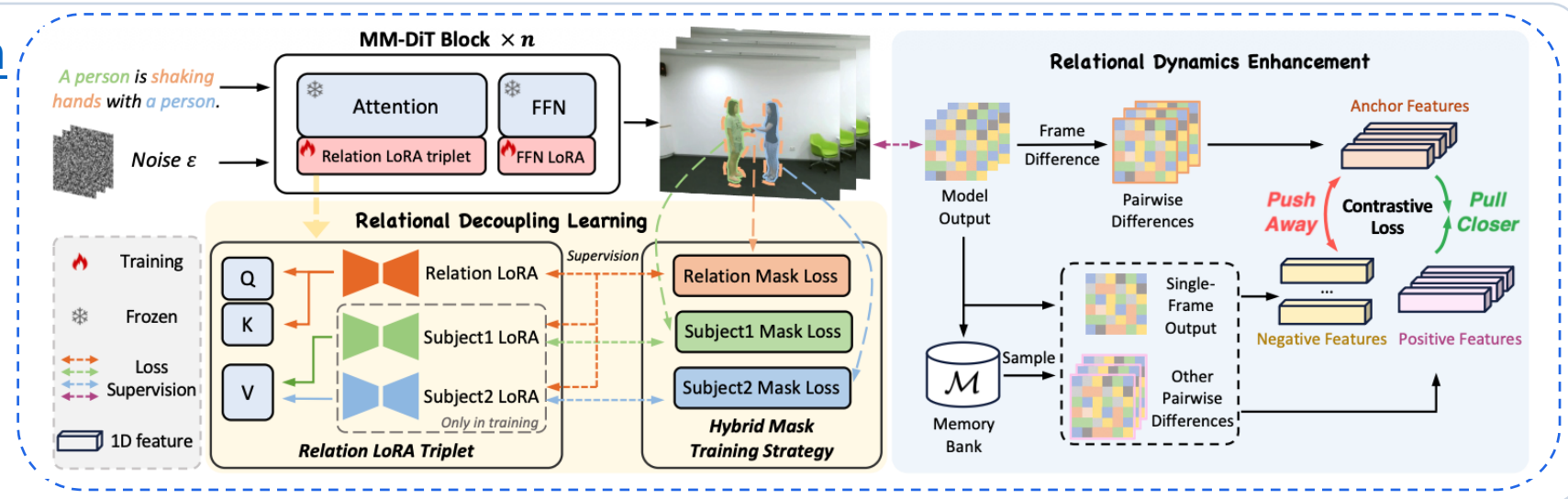
- Head-level DiT decoupling
- 三阶段: Head 分类 + Spatial LoRA + Temporal LoRA



- Training-free motion transfer 加速
- 减少 motion / gradient redundancy 提升效率

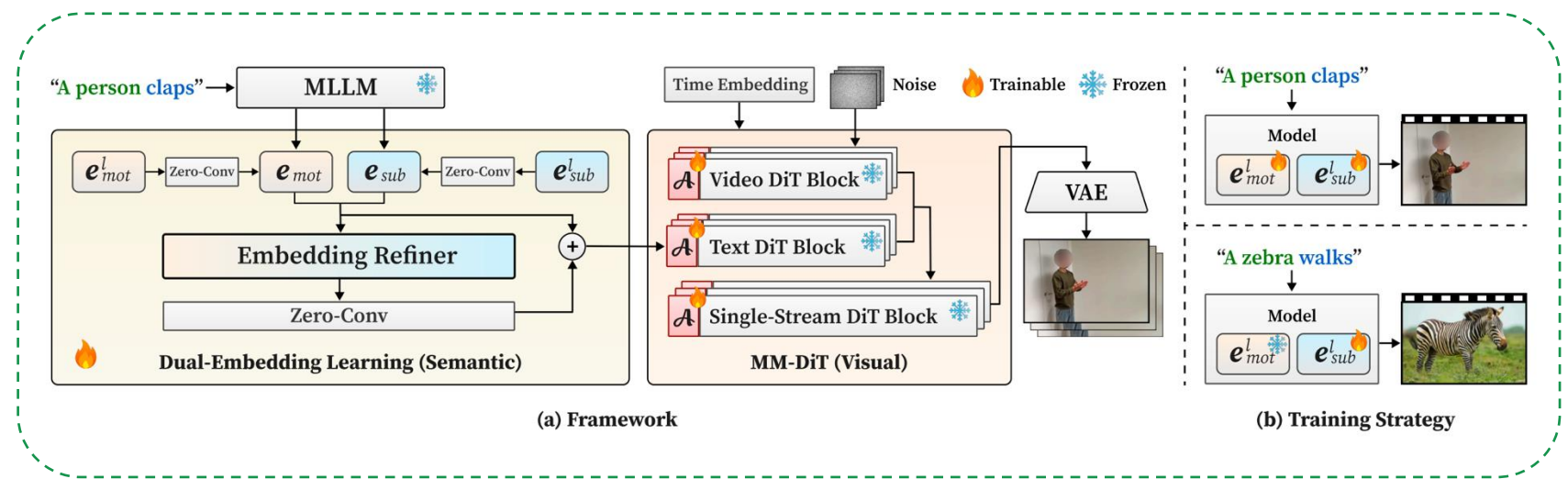
2025 ICCV DreamRelation

- **Relational Decoupling Learning**
 - relation LoRA triplet
 - hybrid mask training strategy (HMT)
- **Relational Dynamics Enhancement**
 - spacetime relational contrastive loss (RCL)



2026 CVPR SynMotion

- **At the semantic level**
 - Dual-Embedding Semantic Comprehension
- **At the visual level**
 - Motion-Aware Adapter
- **embedding-specific training strategy**
 - 使用 Subject Prior Videos (SPV) 构建辅助训练集。
 - 交替优化动作嵌入和主体嵌入，确保动作特性学习与主体泛化并存



8篇论文的研究归纳

paper	问题（研究者视角）	关键洞察 / 解决思路 共性/跨域 + 控制信号注入
DreamVideo	如何在扩散模型中注入跨帧运动？	显式运动映射 + 控制信号注入
MotionDirector	如何精准定位并控制运动相关特征？	头部别控制 + 运动解耦
CustomTTT	如何在少样本下自适应目标运动？	自适应 LoRA + 任务特定化
DeT	如何高效解耦并编辑 motion / appearance？	双分支解耦 + 可编辑潜空间
SMRABooth	如何以最少的核心进行运动复用？	身份锚定 + 运动迁移
Motion Inversion	如何从单视频反演可复用的运动表示？	运动反演编码 + 可迁移 latent
FOLLOW-YOUR-MOTION	如何在 Wan2.1 中分阶段迁移运动？	头分支 + 空间 LoRA + 时间 LoRA
FASTVMT	如何在数据成本下降帧率时一致性？	蒸馏 + 轻量架构 + 加速训练



这8篇的共同方向：提升 motion 与 appearance 的可分性、可迁移性、可浮现性和效率。

调研结论：后续实验应该看什么？

评估维度



Motion Fidelity (运动保真度)

- 动作逻辑性 / 时序一致性 / 变形完整性



Edit / Text Fidelity (编辑/文本一致性)

- 指令遵循 / 细节可控性 / 语义一致性



Subject / Appearance Fidelity (主体一致性)

- 身份保持 / 外观稳定 / 场景一致性



Efficiency (效率)

- 训练成本 / 推理速度 / 参数与显存开销

Benchmark

常用基准

- DeT Benchmark
- FOLLOW Benchmark
- 自建数据集（需包含多种动作/复杂运动）

常用指标

- FVD / FVD↓
- CLIP-I / CLIP-T↑
- ID Similarity↑
- Warp / Motion Consistency↑

Method	Quantitative Metrics				User Study			
	Text Sim.↑	Motion Fid.↑	Temp. Cons.↑	Time (s)↓	Motion Pres.↓	App.↓	Text Align.↓	Overall↓
Training-free methods								
MOFT (Xiao et al., 2024)	0.286	0.792	0.922	1230	6.883	6.631	5.894	6.639
MotionClone (Ling et al., 2024)	0.302	0.831	0.901	1015	6.283	5.874	6.642	4.192
SMM (Yatim et al., 2024)	0.279	0.932	0.918	775	4.350	5.086	4.205	5.883
DiTFlow (Pondaven et al., 2025b)	0.375	0.807	0.941	712	3.326	2.417	2.215	3.284
Tuning-based methods								
MotionInversion (Jeong et al., 2024b)	0.295	0.831	0.771	2315	5.417	3.295	5.117	5.074
MotionDirector (Zhao et al., 2023b)	0.292	0.896	0.939	3008	2.217	4.208	3.298	2.216
Ours	0.380	0.971	0.976	727	1.123	1.335	1.174	1.132



看什么：不只看“好看”，更要看“动得对、改得对、主体对、算得动”。

FOLLOW-YOUR-MOTION 方法概览

Input



参考视频



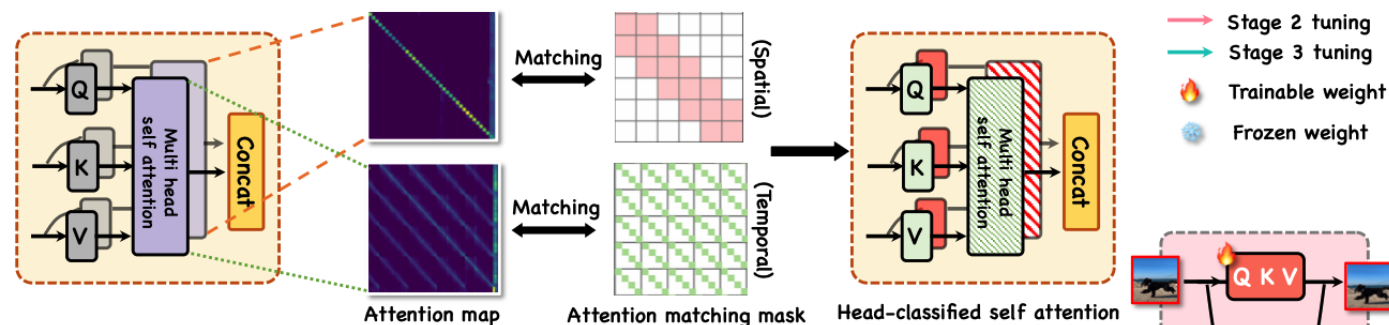
文本 Prompt

"A red and blue truck and a sports car are on the road"

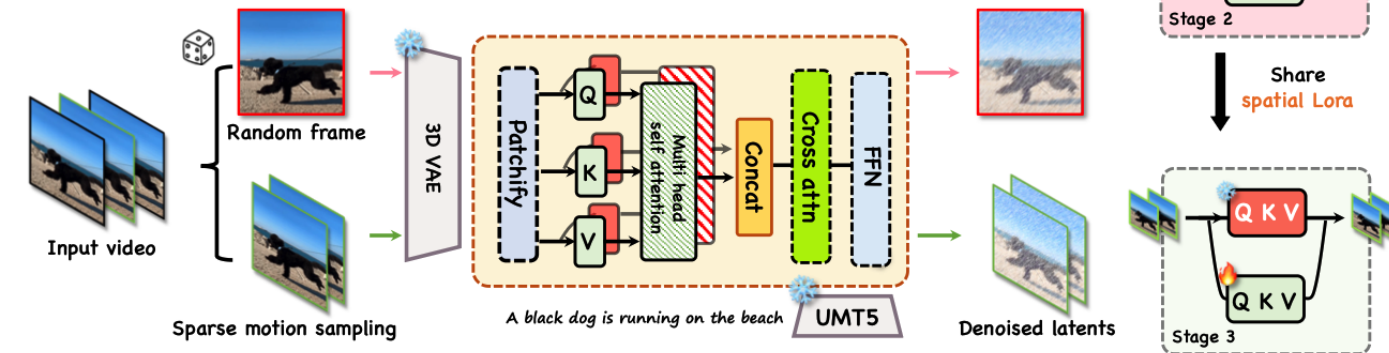


基础模型
Wan2.1

Stage 1: head classification



Stage 2&3: Spatial-temporal decoupled tuning



1 Stage 1
Attention Head
Classification
(头分类)

2 Stage 2
Spatial LoRA
Tuning
(空间 LoRA 微调)

3 Stage 3
Temporal LoRA
Tuning
(时间 LoRA 微调)

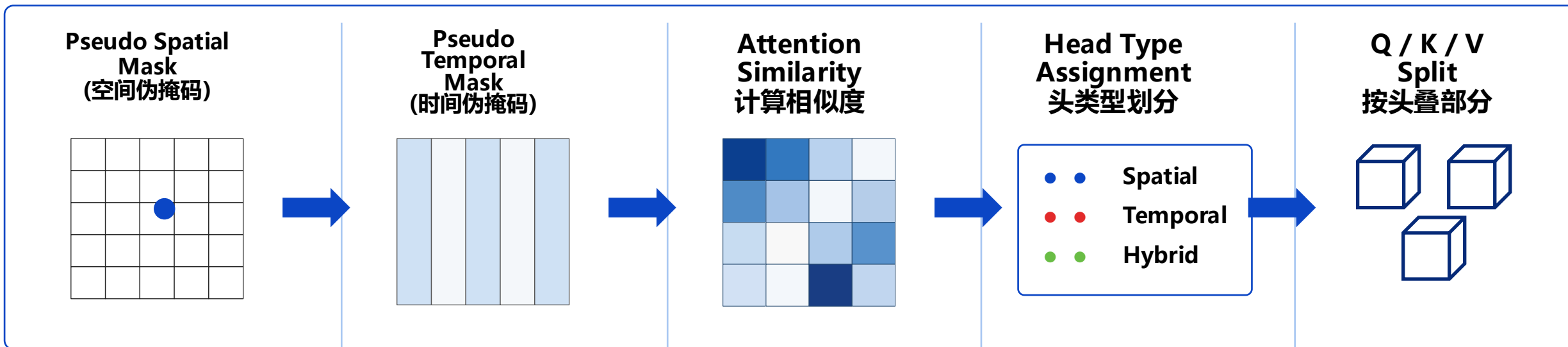
Output

目标视频
(保留运动)



核心思想：先“分类”，再“定空间”，最后“学时间”，实现高效的运动迁移与泛化。

关键实现一： Head Classification 与 Attention Split



```
# 1) 构造空间 / 时间伪掩码
spatial_mask = build_mask("spatial", T, HW)
temporal_mask = build_mask("temporal", T, HW)

# 2) 前向一次 DiT, 收集各层 attention map
_, all_attn = denoising_model(
    noisy_latents, timestep=t,
    context=prompt_emb, **extra_input
)

# 3) 每个 head 分别与两种 mask 计算响应强度
all_head_type = []
for attn in all_attn:          # [B, heads, S, S]
    layer_type = []
    for h in range(num_heads):
        A = attn[:, h]
        ws = (A * spatial_mask).sum()
        wt = (A * temporal_mask).sum()

        # 1 = spatial head, 0 = temporal head
        layer_type.append(1 if wt < 1.3 * ws else 0)

    all_head_type.append(layer_type)

torch.save(all_head_type, "head_types.pt")
```

```
# heads_pos: 0 = temporal, 1 = spatial
def split_QKV(heads_pos):
    self.heads_pos = heads_pos

# 1) 按 head 维度切分原始 Q/K/V 权重
q_t, q_s = split_by_head(self.q.weight, heads_pos)
k_t, k_s = split_by_head(self.k.weight, heads_pos)
v_t, v_s = split_by_head(self.v.weight, heads_pos)

# 2) 重建 temporal / spatial 两套投影层
self.q_temporal = Linear(in_dim, dim_t)
self.q_spatial = Linear(in_dim, dim_s)
self.k_temporal = Linear(in_dim, dim_t)
self.k_spatial = Linear(in_dim, dim_s)
self.v_temporal = Linear(in_dim, dim_t)
self.v_spatial = Linear(in_dim, dim_s)

load_split_weights(...)
del self.q, self.k, self.v
```

```
def forward(x, freqs):
    # 3) 分别计算 temporal / spatial QKV, 再按原 head 顺序拼回
    q = mix_heads(q_temporal(x), q_spatial(x), heads_pos)
    k = mix_heads(k_temporal(x), k_spatial(x), heads_pos)
    v = mix_heads(v_temporal(x), v_spatial(x), heads_pos)

    q, k = rope(q, freqs), rope(k, freqs)
    return self.o(flash_attention(q, k, v))
```



Takeaway: 通过伪掩码 + 相似度度量, 实现 head 级别的活动 / 空间解耦与可控分配。

关键实现二： motion 和 subject / appearance 如何分开处理？

Subject / Appearance 分支 (空间)

随机帧 latent (代表外观)



Spatial LoRA 训练

静态特征潜量损失
(重建外观与场景)

Motion 分支 (时间)

全视频 latent (代表运动)



Temporal LoRA 训练
(冻结 Spatial LoRA)

损失：去噪损失 + 运动一致性损失
(Warp / Cycle / Flow Consistency)

解耦
协同

```
# EffiVMT_train_wan_t2v_Head.py
# Spatial: 单帧 latent 学 appearance
# Temporal: 全视频 latent 学 motion

def training_step(batch):
    if train_temporal_lora:
        latents = batch["latents"] # full video latent
    else:
        i = randint(len(batch["frames_latents"]))
        latents = batch["frames_latents"][i] # random single-frame latent

    noise = randn_like(latents)
    t = sample_timestep()
    noisy_latents = scheduler.add_noise(latents, noise, t)
    target = scheduler.training_target(latents, noise, t)

    pred = denoising_model(
        noisy_latents, timestep=t,
        prompt_emb=prompt_emb,
        extra_input=extra_input
    )

    loss = MSE(pred, target) # denoising loss

    if train_temporal_lora:
        pred_dt = abs(pred[:, :, 1:] - pred[:, :, :-1])
        target_dt = abs(target[:, :, 1:] - target[:, :, :-1])
        loss += 1 - cosine_similarity(pred_dt, target_dt).mean()

    return loss * scheduler.training_weight(t)
```

```
# Temporal stage:
# 先加载并冻结 Spatial LoRA, 再只训练 Temporal LoRA
```

```
inject_adapter(
    target_modules=[
        "self_attn.q_spatial",
        "self_attn.k_spatial",
        "self_attn.v_spatial"
    ],
    adapter_name="spatial_lora"
)
freeze("spatial_lora")

inject_adapter(
    target_modules=[
        "self_attn.q_temporal",
        "self_attn.k_temporal",
        "self_attn.v_temporal"
    ],
    adapter_name="temporal_lora"
)
```

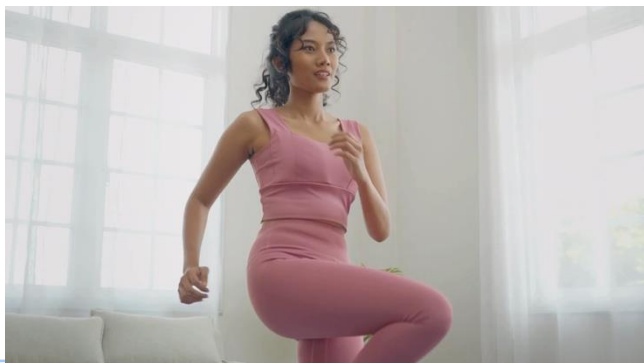
```
load_state_dict(pretrained_spatial_lora_path)
```



Takeaway: 用空间分支学习 “静止世界”，用时间分支学习 “如何动”，解耦建模，协同优化。

复现结果

参考视频 (Reference)



生成视频 (FOLLOW 复现结果)



复现设置

Base Model	Wan2.1-T2V-14B
Input	参考视频 + Prompt
Output	生成目标视频 (保留运动)
Current Status	复现运行中 (空间 LoRA 完成, 开始 时间 LoRA)
Next Experiments	完善指标评测 + Ablation 实验 + 泛化 测试

消融 / 对照实验设计 (建议)

1	2	3	4
Vanilla Wan2.1 (无 LoRA)	Spatial-only (仅空间 LoRA)	Temporal-only (仅时间 LoRA)	Full FOLLOW (空间 + 时间)
baseline 直接生成	只学外观结构	只学运动模式	完整方法 (我们的复现)



结论：复现不应停留在 “生成了一个视频”，而应验证 motion transfer 是否真正成立。